IP: Custom 7x7 Image Filter v1.0

Introduction

The Custom 7x7 Image Filter IP is a high-performance hardware accelerator for the Xilinx Zynq-7000 SoC (Zedboard). It implements a spatial filtering engine with a programmable 7x7 kernel, offloading pixel-intensive convolution tasks from the Processing System (PS) to Programmable Logic (PL). The core features a pipelined "sliding window" architecture, enabling single-cycle throughput per pixel.

Features

- **High-Performance**: Parallel Multiply-Accumulate engine processes 49 taps per clock cycle.
- **Programmable Kernel**: Full 7x7 coefficient matrix configurable via software.
- **Optimized Bandwidth**: "Column-Update" architecture reduces AXI write traffic by 7x.
- **Mixed-Precision**: 8-bit Unsigned Input/Output; 16-bit Signed Coefficients.
- **Approximation Divider**: Efficient Multiplier-Shift logic replaces hardware division.
- **Output Stage**: Configurable Absolute Value (ABS) and Saturating Clamp (0-255).
- **Interface**: Standard AXI4-Lite Slave.



IP Facts Table

Core Specifics	
Supported Device	Xilinx Zynq-7000 (Zedboard)
Interface	AXI4-Lite Slave (32-bit Data Width)
Clock Frequency	100 MHz (Max)
Throughput	1 Pixel / Clock Cycle (Pipeline)
Latency	1 Clock Cycle (Single-Cycle Execution)

Example Use Case: 3x3 Box Blur Verification

Application: Verifying functionality using a standard 3x3 Box Blur.

1. Kernel Configuration:

Load a 3x3 kernel of '1's into the center of the 7x7 matrix. Sum of coefficients = 9.

2. Divider Setup (Divide by 9):

- **Multiplier (M):** $\frac{2^{16}}{9} \approx 7282.$
- **Shift (S):** 16.
- **Hardware Op:** $Y_{out} = (Y_{raw} \times 7282) \gg 16$

3. Verification:

Streaming a flat field of pixels with value 10 results in:

- **Raw Sum:** $9 \times 10 = 90$
- **Calculation:** $(90 \times 7282) \gg 16 = 10$
- **Result:** RESULT_REG reads 0x0A (10), confirming correct operation

Functional Description

Sliding Window Engine

The core maintains an internal 7x7 pixel window (49 pixels). A 2D shift register architecture allows the PS to update the window by writing only the new column of 7 pixels to the DATA_IN_2 register. This triggers a hardware shift and calculation cycle, minimizing bus latency.

Arithmetic Pipeline

The filter performs a weighted sum of the 49 pixels:

$$Y_{raw} = \sum_{i=0}^{7} \sum_{j=0}^{7} (Pixel_{i,j} \times Coeff_{i,j})$$

The result is stored in a 32-bit signed accumulator. Normalization is achieved via a multiplier-shift approximation:

$$Y_{out} = (Y_{raw} \times M) \gg S$$

where M and S are user-configurable.

Post-Processing

1. **ABS Mode:** Calculates $|Y_{out}|$ for edge detection kernels with negative weights.
2. **Clamp:** Saturates output values to the valid 0-255 range.

Register Space

Offset	Name	Bit Field	Description
0x00	**CONTROL_REG**	[0]	**Enable:** 1 = Active.
		[2]	**ABS:** 1 = Enable Absolute Value.
		[3]	**Clamp:** 1 = Enable 0-255 Clamp.
		[12:8]	**Shift:** Right-shift amount (S).
		[31:16]	**Multiplier:** Scale factor (M).
0x08	**DATA_IN_1**	[31:0]	Packed input pixels for Rows 0-3 of the new column.
0x0C	**DATA_IN_2**	[23:0]	Packed input pixels for Rows 4-6. **Triggers Calculation.**
0x10	**RESULT_REG**	[7:0]	Final filtered 8-bit pixel output.
0x14 - 0x78	**COEFF_REGS**	[31:0]	Packed 16-bit signed coefficients (K_0 to K_{48}). Even=[15:0], Odd=[31:16].